Decoding via Statistical Inference. To recover the hidden message, we leverage the statistical properties of the watermarking mechanism. We test whether each key k_i was used to perturb the token distribution. The detection follows the methodology from prior watermarking work [24]:

1. The decoder reconstructs the set of boosted and suppressed tokens for each k_i.
2. It analyzes whether the sampled tokens in the observed text align with the expected distribution shifts for a given k_i.
3. Using statistical tests (e.g., a z-test over token frequency deviations), the decoder determines whether the perturbations associated with k_i are present.
4. If the statistical test indicates the presence of perturbations from k_i, we infer $m_i = 1$; otherwise, we infer $m_i = 0$.

Robustness Against Targeted Attacks. An adversary might attempt to remove a hidden message by selectively disrupting certain bits. To defend against such targeted attacks, we use a key-selection strategy. Rather than deterministically cycling through the watermarking keys k_i for bits $m_i = 1$, we dynamically select a key from this set at each covertext token generation step.

Specifically, we maintain a list of keys:

$$K_m = \{k_i \mid m_i = 1\}$$

and use a PRF to randomly select a key from this list for each token position j:

$$k_j = K_m[F_k(j) \mod |K_m|].$$

This selected key k_j is then used in another PRF, salted with the conversation state s, to perturb the probability distribution when generating token T_j:

$$r = F_{k_j}(s, T_{j-c}, \ldots, T_{j-1}).$$

This process ensures that the perturbation is unpredictable while still recoverable by the recipient, as different keys are sampled dynamically while maintaining alignment between sender and receiver. This makes it difficult for an attacker to selectively erase individual bits without corrupting the entire message, as key usage is non-trivial to predict.

Algorithm Overview

Here we give a high-level overview of the encoding and decoding algorithms. Full pseudocode is provided in Algorithm 1 and Algorithm 2 for the encoding process, and in Algorithm 3 for the decoding process.

Encoding Algorithm:

1. Compute salt s from conversation history.
2. For each token T_j:
   (a) Compute k_j
   (b) Compute $r = F_{k_j}(s, T_{j-c}, \ldots, T_{j-1})$.
   (c) Modify the probability distribution over T_j using r.
   (d) Sample T_j from the perturbed distribution.

3. Output the generated text as stegotext.

Decoding Algorithm:

1. Compute boosted and suppressed token lists for each possible watermarking key k_i.
2. Count occurrences of boosted and suppressed tokens in the observed text.
3. Perform statistical tests to determine whether the distribution matches the expected perturbations for k_i.
4. Infer the hidden bit m_i based on statistical confidence.

This watermarking-based approach provides a practical steganographic mechanism for weak robustness. The integration of salting and dynamic key selection weakens adversarial attacks, ensuring that hidden messages remain difficult to detect while remaining recoverable by authorized recipients.

Algorithm 1 Perturb

Input: A probability distribution $\boldsymbol{p} = (p_1, \ldots, p_N)$, a binary vector $\boldsymbol{r} \in \{0,1\}^N$, and a perturbation parameter $\delta \in (0,1)$.

Let $\mathcal{I} \leftarrow \{\, i \in [N] : p_i \in [2\delta, 1-2\delta] \,\}$
Let $w \leftarrow |\{\, i \in \mathcal{I} : r_i = 1 \,\}|$
Compute $\delta' \leftarrow \dfrac{\delta\, w}{N - w}$
for all $j \in \mathcal{I}$ **do**
    if $r_j = 1$ **then**
        Set $p'_j \leftarrow p_j + \delta$
    else
        Set $p'_j \leftarrow p_j - \delta'$
    end if
end for
for all $j \in [N] \setminus \mathcal{I}$ **do**
    Set $p'_j \leftarrow p_j$
end for
return $\boldsymbol{p'} = (p'_1, \ldots, p'_N)$

Algorithm 2 Encode

Input: key $k = (k_1, \ldots, k_\ell)$, history $\boldsymbol{h}$, message to be encoded $m \in \{0,1\}^\ell$, hardness parameter δ

Compute the salt s from the history $\boldsymbol{h}$.
for $j = 1, 2, \ldots$ **do**
    Sample $i \leftarrow \{j \in [\ell] \ : \ m_j = 1\}$ using a PRF
    Apply the language model over previous tokens to get a probability distribution $\boldsymbol{p}$ over the tth token
    Compute $\boldsymbol{r} \leftarrow F_{k_i}(s, T_{j-c}, \ldots, T_{j-1})$, where $T_{j-c}, \ldots, T_{j-1}$ are the c prior tokens
    Compute the perturbed distribution $\boldsymbol{p'} \leftarrow \mathsf{Perturb}(\boldsymbol{p}, \boldsymbol{r}, \delta)$.
    Sample the next token T_j using the distribution $\boldsymbol{p'}$.
end for
return the generated text as the stegotext $st \leftarrow T_1 T_2 \ldots$

Proofs. Proofs are given in Appendix B.

Algorithm 3 Decode

Input: key $k = (k_1, \ldots, k_\ell)$, history $\boldsymbol{h}$, stegotext $ct = T_1 T_2 \ldots$, threshold parameter z

Compute the salt s from the history $\boldsymbol{h}$.
Initialize counters $ctr_1, \ldots, ctr_\ell \leftarrow 0$
for $j = 1, 2, \ldots$ **do**
  for $i = 1, \ldots, \ell$ **do**
    Compute $\boldsymbol{r} \leftarrow F_{k_i}(s, T_{j-c}, \ldots, T_{j-1})$, where $T_{j-c}, \ldots, T_{j-1}$ are the c prior tokens
    Let $q \in \{0, 1, \cdots, N-1\}$ denote the index of T_j within the vocabulary. If $r_q = 1$, increment $ctr_i \leftarrow ctr_i + 1$
  end for
end for
For each $i \in [\ell]$, if $ctr_i \geq z$, set $m_i \leftarrow 1$. Otherwise, set $m_i \leftarrow 0$
return $m = m_1 m_2 \cdots m_\ell$

6.2 Embedding Scheme for Strong Robustness

To achieve strong robustness, we leverage text embeddings and a locally sensitive hash (LSH) function to encode messages in a way that is resilient to paraphrasing attacks. The core idea is to use rejection sampling to generate stegotext whose embedding hashes to the desired message bits. We assume that the LSH is keyed in the following explanation or that it is used on already encrypted data to ensure confidentiality.

Main Idea. The embedding scheme encodes a message $m \in \{0,1\}^n$ by breaking it into chunks of l bits each. For each chunk, rejection sampling is applied during text generation to ensure that the embedding of the generated stegotext maps to the corresponding l-bit chunk under the LSH. This process continues iteratively until all n bits of the message are encoded. The decoding process reconstructs m by computing the LSH of the received text embeddings and concatenating the resulting l-bit hashes.

Encoding. The encoding algorithm ensures that the generated text's embedding satisfies the desired hash values. The process is as follows:

1. **Initialization:** Break the message m into $\lceil n/l \rceil$ chunks, $m_1, m_2, \ldots, m_{\lceil n/l \rceil}$, where each chunk $m_i \in \{0,1\}^l$.
2. **Chunk-wise Generation:** For each chunk m_i:
   - Generate a candidate text x_i using the LLM.
   - Compute the embedding $E(x_i)$ of the generated text.
   - Apply the LSH function $H(E(x_i))$ to the embedding.
   - If $H(E(x_i)) = m_i$, accept x_i and move to the next chunk. Otherwise, repeat the process with a new candidate text.
3. **Output:** Concatenate the accepted texts $x_1, x_2, \ldots$ to form the stegotext st.

Decoding. The decoding algorithm reconstructs the hidden message m from the received stegotext st:

1. **Chunk-wise Decoding:** Break the stegotext st into the same chunks $x_1, x_2, \ldots$ used during encoding.
2. **Hash Extraction:** For each chunk x_i, compute its embedding $E(x_i)$ and apply the LSH function $H(E(x_i))$ to extract the l-bit hash.
3. **Message Reconstruction:** Concatenate the hashes of all chunks to reconstruct m.